CEG 3156: Computer Systems Design
(Winter 2026)

Prof. Rami Abielmona

Lab #3:
Pipelined MIPS Processor

Submitted by:
Mann Patel — 300363879
Surya Vasudev — 300358602

Date: April 8th, 2026

Table of Contents

1. Introduction . . . 3
2. Design Overview . . . 3
3. Component Design . . . 3
   3.1. Pipeline Registers . . . 3
   3.2. Control Unit (`Control_Unit`) . . . 4
   3.3. ALU and ALU Control . . . 4
   3.4. Instruction Memory (`instruction_memory`) . . . 4
   3.5. Data Memory (`data_memory`) . . . 4
   3.6. Register File (`register_file`) . . . 4
   3.7. Sign Extension Unit (`sign_extended`) . . . 4
   3.8. Branch and Jump Resolution . . . 5
   3.9. Output Debug Multiplexer . . . 5
4. Benchmark Program and Cycle Trace . . . 5
5. Timing Analysis and CPU Execution Time . . . 5
   5.1. Critical Path Analysis . . . 5
      5.1.1. Gate-Level Delay of the 8-Bit Ripple-Carry Adder . . . 6
      5.1.2. ALU Worst-Case Delay . . . 6
   5.2. CPI and CPU Execution Time . . . 6
   5.3. Comparison with Single-Cycle Processor . . . 7
6. Simulation Results . . . 7
7. Design Challenges and Solutions . . . 8
8. Conclusion . . . 8

1. Introduction

The objective of this laboratory is to design, implement, and verify a pipelined MIPS processor in VHDL. The processor is built on top of the single-cycle processor from Lab 2 and supports the same subset of the MIPS instruction set:

- **Memory-reference instructions:** `lw` (load word) and `sw` (store word)
- **Arithmetic/logic instructions:** `add`, `sub`, `and`, `or`, and `slt`
- **Control-flow instructions:** `beq` (branch if equal) and `j` (unconditional jump)

The design keeps the same **8-bit data paths** and **32-bit instruction widths** from Lab 2. The key addition is a five-stage pipeline with four pipeline registers, a forwarding unit to resolve data hazards, and a hazard detection unit to handle load-use stalls.

2. Design Overview

The processor divides execution into five overlapping stages, allowing a new instruction to be issued every clock cycle once the pipeline is full:

1. **IF (Instruction Fetch):** The PC addresses instruction memory and PC+1 is computed for the next cycle.
2. **ID (Instruction Decode):** The fetched instruction is decoded, source registers are read, and all control signals are generated.
3. **EX (Execute):** The ALU performs its operation. Forwarding muxes select the correct operands. The branch target address is calculated.
4. **MEM (Memory Access):** Data memory is read or written. The branch outcome is resolved here.
5. **WB (Write Back):** The ALU result or memory read data is written back to the register file.

Between each adjacent pair of stages sits a pipeline register (IF/ID, ID/EX, EX/MEM, MEM/WB) that captures all data and control signals needed by downstream stages. A forwarding unit and a hazard detection unit are added to keep the pipeline correct in the presence of data and control hazards.

3. Component Design

3.1. Pipeline Registers

Four pipeline registers partition the datapath. Each is built entirely from `dFF_reset` flip-flops and `reg_8` registers and carries both data and control signals forward.

Register	Contents
IF/ID	32-bit instruction, PC+4. Supports stall (load-enable) and flush (reset).
ID/EX	All nine control signals, PC+4, ReadData1, ReadData2, sign-extended immediate, jump address, rs/rt/rd register numbers, funct field.
EX/MEM	MemtoReg, RegWrite, MemRead, MemWrite, Branch, Jump, branch target, jump address, Zero flag, ALU result, ReadData2, destination register.
MEM/WB	MemtoReg, RegWrite, memory read data, ALU result, destination register.

Table 1: Pipeline register contents.

All pipeline registers assert `int_reset = i_reset OR i_flush` so that a global reset and a pipeline flush share the same synchronous reset path.

Instr.	RegDst	ALUSrc	Mem-toReg	RegWrite	Mem-Read	MemWrite	Branch	Jump	ALUOp
R-type	1	0	0	1	0	0	0	0	10
`lw`	0	1	1	1	1	0	0	0	00
`sw`	×	1	×	0	0	1	0	0	00
`beq`	×	0	×	0	0	0	1	0	01
`j`	×	×	×	0	0	0	0	1	××

Table 2: Control signal truth table.

3.2. Control Unit (`Control_Unit`)

The control unit is unchanged from Lab 2. It decodes the 6-bit opcode and produces nine control signals. In the pipelined design, these signals enter the ID/EX register immediately after being generated and travel with the instruction through the pipeline.

When a load-use stall is active, all nine control signals are AND-gated with `NOT haz_ctrl_flush` before entering the ID/EX register, effectively inserting a NOP bubble without disrupting the instruction already in ID/EX.

3.3. ALU and ALU Control

The ALU and ALU control units are identical to Lab 2. The ALU control decodes `ALUOp` combined with the `funct` field to produce a 3-bit operation code. The 8-bit ALU supports AND, OR, addition/subtraction, and set-less-than, and produces a `Zero` flag used for branch resolution.

In the pipelined design, `ALUOp` and `funct` travel through the ID/EX register. The forwarding muxes sit between the ID/EX outputs and the ALU inputs, selecting the correct operand from the register file, the MEM/WB write-back path, or the EX/MEM ALU result.

3.4. Instruction Memory (`instruction_memory`)

The instruction memory is a 256×32 LPM ROM, identical to Lab 2. It is clocked on the inverted clock (`NOT GClock`) so that the address is latched on the falling edge and the 32-bit instruction output is combinatorially available before the next rising edge.

3.5. Data Memory (`data_memory`)

The data memory is a 256×8 LPM RAM DQ. It is also clocked on the inverted clock for the same timing reason. Writes are registered on the falling edge; reads are combinatorial. The `wren` input is driven by `exmem_ctrl_MemWrite`.

3.6. Register File (`register_file`)

The register file is unchanged from Lab 2: eight 8-bit registers with two read ports and one write port, with register 0 hardwired to zero. In the pipelined design, writes are driven by MEM/WB stage signals (`memwb_ctrl_RegWrite`, `memwb_write_reg`, `wb_write_data`), while reads are driven by the IF/ID instruction fields.

3.7. Sign Extension Unit (`sign_extended`)

The sign extension unit passes the lower 8 bits of the 16-bit immediate field through to the 8-bit datapath, identical to Lab 2.

3.8. Branch and Jump Resolution

Branches are resolved in the MEM stage. The signal `mem_take_branch = exmem_ctrl_Branch AND exmem_zero` selects the branch target over the sequential PC and triggers a pipeline flush, cancelling the two instructions fetched speculatively after the branch.

Jumps are resolved in the ID stage. The lower 8 bits of the instruction word form the jump target address. When `id_ctrl_Jump_s` is asserted, `pipeline_flush` is set, flushing the IF/ID register. Both flush sources are combined as:

```
pipeline_flush = mem_take_branch OR id_jump_taken
```

3.9. Output Debug Multiplexer

The debug multiplexer is identical in function to Lab 2. `ValueSelect[2:0]` selects among PC, ALU result, ReadData1, ReadData2, WriteData, and a packed control information word. The control word carries `RegDst`, `Jump`, `MemRead`, `MemtoReg`, `ALUOp[1:0]`, and `ALUSrc` sourced from the ID/EX register.

ValueSelect	MuxOut	Description
`000`	`PC[7:0]`	Current program counter
`001`	`ALUResult[7:0]`	EX stage ALU result
`010`	`ReadData1[7:0]`	Register file read port 1
`011`	`ReadData2[7:0]`	Register file read port 2
`100`	`WriteData[7:0]`	WB stage write data
other	Control word	`{0, RegDst, Jump, MemRead, MemtoReg, ALUOp, ALUSrc}`

Table 3: Output multiplexer selection.

4. Benchmark Program and Cycle Trace

The following benchmark was loaded into instruction memory. Data memory was pre-initialized with `mem[0x00] = 0x55` and `mem[0x01] = 0xAA`.

5. Timing Analysis and CPU Execution Time

5.1. Critical Path Analysis

In the pipelined design, the clock period is determined by the slowest individual stage rather than the total datapath. Examining each stage:

Stage	Delay (ps)	Limiting component
IF	200	Instruction memory read
ID	100	Register file read
EX	100	8-bit ALU (80 ps adder + 20 ps mux overhead)
MEM	200	Data memory read or write
WB	100	Register file write

Table 5: Per-stage delay breakdown.

The slowest stages are IF and MEM at 200 ps each, giving:

Cycle	PC	Instruction	ALU Result	Zero	Branch	MemW	RegW	Hazard
1	00	`lw $2, 0`	00	0	0	0	1	None
2	01	`lw $3, 1`	01	0	0	0	1	None
3	02	`sub $1,$2,$3`	AB	0	0	0	1	Load-use stall on $2
4	03	`or $4,$1,$3`	FF	0	0	0	1	EX/MEM forward on $1
5	04	`beq $1,$1,20`	00	1	1	0	0	Control hazard, 2-cycle flush
6	05	`sw $4, 3`	–	–	–	–	–	Flushed
7	06	`add $1,$2,$3`	FF	0	0	0	1	Branch target
8	07	`sw $1, 4`	04	0	0	1	0	EX/MEM forward on $1
9	08	`lw $2, 3`	03	0	0	0	1	None
10	09	`lw $3, 4`	04	0	0	0	1	None
11	0A	`j 11`	–	–	0	0	0	Jump, 1-cycle flush
12	0B	`beq $1,$1,-44`	–	–	–	–	–	Flushed
13	11	`beq $1,$2,-8`	00	1	1	0	0	Branch taken, 2-cycle flush

Table 4: Cycle-by-cycle execution trace of the benchmark program.

$$T_{\text{clock}} = 200 \text{ ps}$$

$$f_{\max} = \frac{1}{200 \text{ ps}} = 5 \text{ GHz}$$

5.1.1. Gate-Level Delay of the 8-Bit Ripple-Carry Adder

Each 1-bit full adder has a carry propagation of one gate delay (0.01 ns). For 8 cascaded stages:

$$t_{\text{adder}} = 8 \times 0.01 \text{ ns} = 0.08 \text{ ns} = 80 \text{ ps}$$

5.1.2. ALU Worst-Case Delay

$$t_{\text{ALU}} = t_{\text{adder}} + t_{\text{mux}} = 80 \text{ ps} + 20 \text{ ps} = 100 \text{ ps}$$

5.2. CPI and CPU Execution Time

An ideal pipeline achieves CPI = 1. Hazards add stall cycles:

Hazard	Penalty (cycles)	Cause
Load-use stall (lw then sub)	1 to 2	Loaded value not ready for next instruction
Branch taken (resolves in MEM)	2	Two speculatively fetched instructions flushed
Jump (resolves in ID)	1	One speculatively fetched instruction flushed

Table 6: Hazard penalty summary for the benchmark program.

For 13 instructions with approximately 5 to 6 extra stall cycles:

$$\text{CPI}_{\text{pipelined}} \approx \frac{13 + 6}{13} \approx 1.46$$

$$T_{\text{CPU}} = \text{IC} \times \text{CPI} \times T_{\text{clock}} = 13 \times 1.46 \times 200 \text{ ps} \approx 3796 \text{ ps} \approx 3.8 \text{ ns}$$

5.3. Comparison with Single-Cycle Processor

The single-cycle processor from Lab 2 had a clock period equal to the full critical path:

$$T_{\text{cycle,single}} = 200 + 100 + 100 + 200 + 100 = 700 \text{ ps}$$

$$T_{\text{CPU,single}} = 11 \times 1 \times 700 \text{ ps} = 7700 \text{ ps} = 7.7 \text{ ns}$$

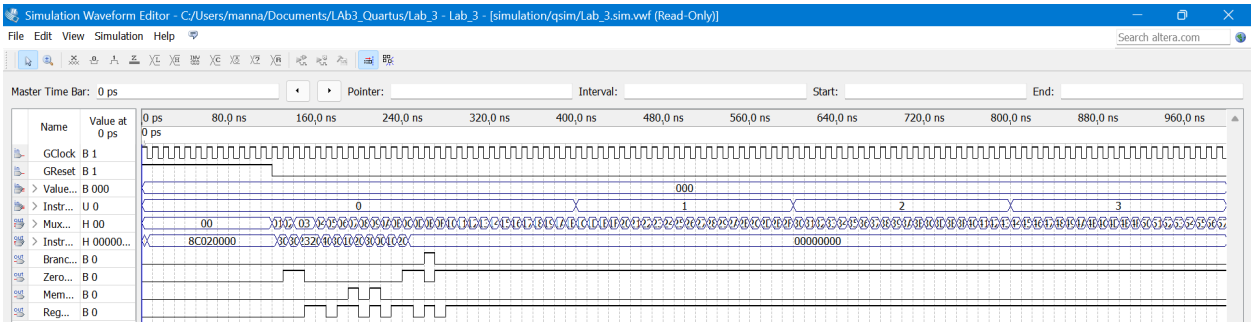
The pipelined processor is approximately **2× faster** on this benchmark. Without any hazard penalties the theoretical speedup would be 3.5×, which shows how significantly stalls and flushes reduce the benefit of pipelining in short programs.

6. Simulation Results

Functional simulation was performed in Quartus II 13.1. The simulation verified correct pipelined execution of the benchmark program by observing the following signals at each clock cycle:

- `InstructionOut[31:0]` with varying `InstrSelect`: confirmed that the correct instruction appears in each pipeline stage at the expected cycle.
- `MuxOut[7:0]` with varying `ValueSelect`: confirmed correct PC progression, ALU results, register file read/write data, and control signal values.
- `BranchOut`, `ZeroOut`, `MemWriteOut`, `RegWriteOut`: confirmed correct control signal assertion per instruction.
- Stall and flush behaviour: confirmed that the PC and IF/ID register freeze correctly on a load-use hazard, and that IF/ID and ID/EX are cleared correctly on a taken branch or jump.

Waveform 1 — PC and InstructionOut (InstrSelect = 000)



Waveform 2 — ALU Result and control signals (ValueSelect = 100)

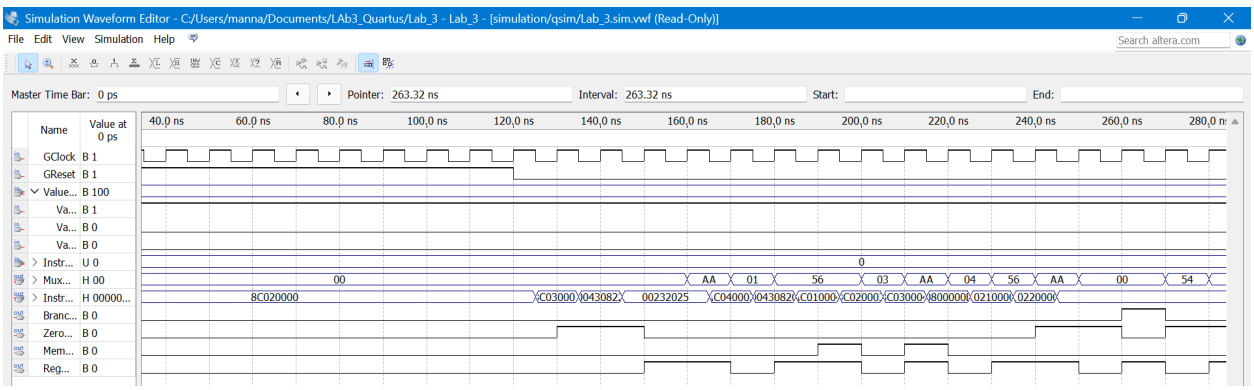


Figure 1: Simulation waveforms (Quartus II functional simulation).

7. Design Challenges and Solutions

Several design obstacles were encountered during implementation:

1. **Coordinating Jump and Branch Flushes:** Jumps resolve in the ID stage, one stage earlier than branches. Both flush sources needed to be combined without either masking the other. This was solved by OR-ing them into a single `pipeline_flush` signal that feeds the flush input of all affected pipeline registers.

2. **Load-Use Stall Interacting with a Flush:** A load-use stall and a pipeline flush can coincide in the same clock cycle. Inside each pipeline register, the flush is wired to the synchronous reset input and the stall is wired to the load-enable input. Synchronous reset takes priority over load-enable in the `reg_8` and `dFF_reset` components, so the flush always wins cleanly without additional arbitration logic.

8. Conclusion

A fully functional 8-bit pipelined RISC processor was designed, implemented in structural VHDL, and verified in simulation. The processor executes all supported MIPS instructions across a five-stage pipeline with forwarding and hazard detection.

The critical path analysis gives a maximum theoretical clock frequency of 5 GHz (200 ps clock period), constrained by the instruction and data memory stages. The CPU execution time for the benchmark program is approximately 3.8 ns, representing roughly a 2× improvement over the single-cycle processor from Lab 2. The theoretical maximum speedup without hazards would be 3.5×, and the gap between that and the

achieved speedup reflects the cost of load-use stalls and branch/jump flush penalties in this short benchmark.

Note: The complete VHDL source code for all components is included in the GitHub repository: https://github.com/MannP5/CEG3156_LAB_GROUP_4